

Graph-based Retrieval Enhancement for Semi-Structured Data

Dennis Halilaj
M.Sc. Informatics
dennis.halilaj@tum.de

Michaela Blomeier
M.Sc. Robotics,
Cognition, Intelligence
michaela.blomeier@tum.de

Olga Czajkowska
M.Sc. Informatics
olga.czajkowska@tum.de

Supervisor: Jakob Sturm
Social Computing Research Group
jakob.sturm@bmw.de

Abstract

Much of the data we work with is effectively semi-structured once metadata is taken into account. In information retrieval, metadata is often neglected or simply concatenated to the text corpus. We ask how categorical, hierarchical and relational information can improve the retrieval of related documents compared to basic approaches such as BM25 and dense embeddings. While prior work explores this idea in domain-specific systems (e.g., IT ticket resolution), there is no general benchmark suite to study such methods. We therefore identify two diverse datasets, align them to a shared schema and build a benchmark suite to evaluate structure-aware retrieval. We implement methods inspired by the literature as well as one of our own, showing how graph-based approaches and relational use of metadata can enhance retrieval on semi-structured data. Our results show that metadata substantially improves retrieval when used relationally and that flattening metadata into text is not an effective substitute for structure. We also provide a benchmark suite and pipeline to support future work and more generalizable results.

1 Introduction

Information retrieval ranks documents by relevance to a query (Manning et al., 2008). Classical models rely on lexical similarity, while dense retrievers embed queries and documents and rank them by semantic similarity (Xu et al., 2026). While both approaches perform well for flat-text documents, many real-world objects also include categorical attributes, nested metadata and explicit links to other objects (Wu et al., 2024). We refer to such objects as **semi-structured data**.

Issues from platforms such as GitHub and scientific papers are typical examples: issues contain text but also labels and links to other issues or pull requests; papers contain text but also author and venue metadata and citation relations. Some metadata is

categorical and some is hierarchical (e.g., methods belonging to broader method families and fields). A common workaround is to flatten metadata into a single text string, but this discards structural signals that could improve retrieval (Yousuf et al., 2026). This motivates **structure-aware retrieval**, which aims to preserve and exploit hierarchies, categories and links (often naturally forming a graph) to retrieve more precisely related documents.

Project Goal and Research Question

Our goal is to test whether the structure present in semi-structured documents improves retrieval beyond flat lexical and dense baselines. We study the following research question:

How can categorical, hierarchical and relational information be used to improve the retrieval of related documents from semi-structured datasets compared with flat lexical and dense-retrieval baselines?

We evaluate a progression of methods with increasing use of structure. BM25 serves as a lexical baseline operating on flattened text, which includes structural content only implicitly and without field-aware modeling (Robertson and Zaragoza, 2009a). A Sentence-Transformer baseline similarly ranks candidates using cosine similarity over embeddings of the flattened text (Reimers and Gurevych, 2019). We compare these to field-aware and graph-based approaches: a CHARM-inspired weighted multi-field encoder (Section 4.1), SAGE-inspired graph expansion (Section 4.2), learned score propagation (Section 4.3) and an attention-based propagation variant (Section 4.4).

2 Background and Related Work

This section presents the foundations and related work to the investigated retrieval approaches of

section 4, progressing from lexical and dense retrieval to field-aware, hierarchical and graph-based methods.

2.1 Retrieval

An information retrieval system receives a query q , assigns a score $s(q, d)$ to every candidate document d and returns the candidates in decreasing score order. The performance of such a retrieval system is commonly evaluated using the standard metrics of Precision@k and Recall@k and also rank-sensitive metrics such as Mean Reciprocal Rank (MRR), Mean Average Precision (MAP) and Normalized Discounted Cumulative Gain (NDCG) (see 5.1). Lexical retrieval methods such as BM25 rank documents primarily based on term overlap and term-weighting statistics, which can be highly effective when relevant documents use the same vocabulary as the query. However, they are less effective when relevance depends on semantic similarity rather than exact lexical matching (Robertson and Zaragoza, 2009b; Karpukhin et al., 2020). Dense retrieval methods mitigate this issue by learning low-dimensional continuous representations of queries and documents in a shared embedding space, such that semantically related items can obtain high similarity scores even without direct token overlap (Karpukhin et al., 2020). Reimers and Gurevych (2019) demonstrates the effectiveness of learned dense representations over lexical term matching for retrieval tasks.

Both lexical and dense retrieval primarily rely on textual content and generally do not explicitly use structural relations within or between documents. Such relations can be incorporated directly in the retrieval or a re-ranking stage and provide the basis for structure-aware and graph-based retrieval approaches.

2.2 Hierarchical and Semi-structured retrieval

Earlier work on structured retrieval primarily exploited the structure of the document itself. For example XML retrieval (Schütze et al., 2008) represents terms together with their structural context which allows distinguishing same terms appearing in different hierarchical positions in the document. More recent approaches also consider structure within the information represented by a document, rather than only the structure of the document itself. For example Freymuth et al. (2025) organizes product descriptions into separate fields according

to information granularity ranging from coarse to increasingly detailed representations. For efficient product retrieval, aggregated representations of the subfields are used and the initial retrieved documents are then reranked based on representations of the more detailed descriptions.

A similar approach is taken in 4.1. Rather than using a separate reranking stage, the hierarchical representations are combined directly using static weighting. This provides structure-aware retrieval behaviour while still remaining practical enough for a lightweight retrieval pipeline.

These results show that exploiting the internal structure of semi-structured data adds retrieval signals beyond textual similarity; we leverage this for both scientific papers and GitHub issues, where inherent metadata complements lexical and dense representations.

2.3 Graph propagation methods and GNN enhanced retrieval

Graph-based retrieval extends traditional text retrieval and hierarchical retrieval methods by incorporating relationships between documents as an additional relevance signal. Documents can be represented as nodes of a graph, while edges are explicit relations, such as citations or issue links or relations derived from shared metadata and hierarchical structure. This allows retrieval systems to identify additional relevant candidates that may not have high textual similarity to the query but are closely related to initially retrieved documents and therefore potentially relevant as well.

SAGE, which is Graph-based neighborhood expansion as introduced in Titiya et al. (2026), preconstructs in an offline phase a graph from the document corpus. During the initial retrieval phase a set of seed documents are retrieved after which the graph neighborhoods of these seed nodes are explored and filtered using dense and sparse query similarity to obtain the final list of retrieval candidates.

An alternative to fixed graph expansion are graph neural networks which update a node’s representation by aggregating information from the node’s local neighborhood through message passing, such as done in Kipf and Welling (2017) with Graph Convolutional Networks, or GraphSAGE (Hamilton et al., 2017) with learnable neighborhood aggregation. It is important to note here that SAGE and GraphSAGE refer to two different methods: SAGE relies on expansion of the graph for retrieval-

ing nodes, whereas GraphSAGE is a Graph Neural Network with a specific neighborhood aggregation formula. GNN-Ret (Li et al., 2024) similarly applies message passing. Instead of passing node embeddings, it does a passing of scores to learn the reweighting of relevance scores of the neighbors of seed documents relevant to the query. Therefore the inclusion of neighbor nodes into the relevance ranking is learned instead of hardcoded with rules, as is done in SAGE (Titiya et al., 2026).

Graph Attention Networks further apply the attention mechanism to graphs and allow for weighting contributions of individual neighbors through the learned attention coefficients (Veličković et al., 2018).

Overall the previous work suggests three strategy paths for utilizing graph structure in retrieval: explicit graph expansion, learned embedding propagation and learned relevance score propagation. Our work evaluates graph expansion and score propagation (see 4.2, 4.3 and 4.4) on graphs derived from scientific papers and GitHub issues for comparing the contribution of graph information to text-based retrieval baselines. We avoid high-dimensional document embedding propagation to keep computation and memory lightweight and focus on scalar query dependent relevance score propagation.

3 Dataset and Preprocessing

To the best of our knowledge, no semi-structured benchmark yet enables comprehensive cross-domain retrieval assessment. We therefore provide a benchmark and preparation pipeline yielding two semi-structured corpora, GitHub issues and scientific papers, each retaining structured metadata to complement textual representations.

3.1 Dataset Sources

The GitHub data is ingested through our pipeline, which downloads and normalizes issue records from a Kaggle-backed GitHub issues dataset (Alshara et al., 2023; Alshara, 2022). The dataset was enriched with links between issues resolved by the same pull request, which served as additional metadata and were not explicitly provided in the original dataset. The GitHub dataset comprises approximately 50,000 links connecting 35,000 issues to 50,000 pull requests, drawn from 900,000 public Android projects on Github. This means that, for each issue used as a query, the issues linked through the same resolving pull request serve as the ground

truth.

For the scientific-paper experiments, we use datasets provided by LitSearch (Group, 2026) and augment them with metadata from Semantic Scholar. LitSearch is a retrieval benchmark containing 597 realistic literature search queries about recent ML and NLP papers. It is constructed using a mix of questions generated by GPT-4 based on paragraphs containing citations to other papers, combined with questions manually written by papers’ authors. To ensure quality, all LLM-generated questions were audited and adjusted by experts (Ajith et al., 2024). Queries are annotated with metadata on source, specificity and level of quality, though we did not use the attributes in our workflow. The corpus contains 64,813 documents with titles, abstracts and outgoing citation paper IDs. However, this dataset lacks much of the metadata we require, so enrichment was needed (see 3.2).

3.2 Dataset Alignment and Enrichment

To enable benchmarking across both GitHub issues and scientific papers, we defined a common JSON schema to which we mapped both datasets to. The schema generalizes domain specific attributes into shared fields:

- unique global document identifiers
- source_dataset and source_type (e.g. issue or paper) metadata
- title, main_text and secondary_texts (e.g. the whole paper, comments on issues)
- structured categorical and hierarchical metadata fields
- explicit relation links for supervised relevance and graph construction

Alignment required considerable preprocessing: mapping all domain identifiers to unique global IDs, merging entries referring to the same document, removing duplicates and standardizing hierarchical label categories. After alignment, the paper corpus was enriched with additional metadata to support hierarchical and graph-based retrieval, since the original entries mainly contained text and citation relations while the proposed methods also require structural attributes. This was not needed for the GitHub dataset, as most hierarchical and categorical fields could already be filled during alignment from the original data.

The paper enrichment therefore encompassed the following three stages:

Enrichment with Semantic Scholar Metadata

Scientific paper documents were first enriched using the Semantic Scholar Academic Graph API (Kinney et al., 2025). For each available paper, metadata including authors, affiliations, publication venue and type, journal information and fields of study were retrieved and mapped to the common dataset schema. The metadata enrichment provides additional structural signals for retrieval and graph construction.

Construction of Citation-Based Paper Queries

To address the absence of paper-to-paper queries in the LitSearch dataset, we constructed these manually from papers containing mapped citation links (up to 200 sampled with a fixed seed). Each selected paper represents a query with its cited papers as the relevant candidates. Such queries are useful for evaluating retrieval of related papers, because topics or methods might be similar without necessarily having high direct textual similarity.

Generation of Hierarchical Metadata

To obtain hierarchical information, the available metadata and textual content of each paper were processed using gpt-5-nano through the OpenAI API¹ and three structured components were returned in JSON format:

- affiliation hierarchy consisting of country, sector and organization the paper belongs to
- a four level field of study hierarchy from broad field to specific topic
- a three level method hierarchy from method family to specific method

Broad hierarchy levels such as field and method family possibilities were predefined in the prompt for the model to pick from, such as to prohibit unrelated classification and generated responses were parsed and evaluated for malformed JSON output or missing entries before incorporation into the aligned data schema.

4 Methods

In this section, we describe the experimental methods implemented to exploit the domain’s categor-

¹<https://developers.openai.com/api/docs/guides/structured-outputs>

ical and hierarchical structure. The approaches range from multi-embedding fusion of hierarchical data to message passing on a self-constructed graph.

4.1 CHARM-inspired Method

This method is a lightweight, structure-aware retrieval approach that separately encodes three document fields: metadata, title and main_text.

Each field is embedded with the frozen all-MiniLM-L6-v2 sentence-transformer model (Reimers and Gurevych, 2019), which was chosen because it performs well on semantic similarity and retrieval tasks, is well established, and remains efficient enough to fit the available resource constraints. The field embeddings are combined using a static weighted average. These field weights are themselves hyperparameters, tuned via random search and manual adjustment (Bergstra and Bengio, 2012).

At query time, the same field extraction is applied. If metadata is present and informative, it is included in the weighted query vector; otherwise the method falls back to using the main text only. The final ranking is based on cosine similarity between the aggregated query vector and the precomputed aggregated document vectors.

The design is inspired by the original Cascading Hierarchical Attention Retrieval Model (CHARM) (Freymuth et al., 2025). CHARM introduces cascading hierarchical attention over multi-field inputs and demonstrates that per-field representation and weighted combination can improve retrieval compared to flat text encodings. The design draws on hierarchical attention ideas but simplifies them, fields are encoded separately, weighted statically, without fine-tuning and aggregated directly across metadata, title and text. This preserves structure-aware retrieval behavior while remaining practical for a common benchmark setting.

4.2 SAGE Graph Expansion-inspired Method

Our graph-expansion approach is inspired by the SAGE Graph Expansion introduced by (Titiya et al., 2026):

In the offline phase we construct for each domain (d), so scientific paper corpus and GitHub issue dataset, a separate multidigraph $G^{(d)} = (V^{(d)}, E^{(d)})$ where each node $v_i \in V^{(d)}$ represents a document or GitHub issue and each edge $e = (v_i, v_j, l_e, w_e) \in E^{(d)}$ represents a direct relation from node v_i to v_j with relation type l_e and

weight w_e . Therefore different types of edges between two nodes can exist and are given different weight depending on the relation type l_e :

- $l_e = \text{explicit relation}$: If two nodes v_i, v_j share a direct citation or the GitHub issues are directly linked, the edge gets a base weight of $w_e = 1.0$.
- $l_e = \text{shared hierarchical path}$: Each enriched document v_i contains hierarchical paths $p_i = (c_1, c_2, \dots, c_L)$ where c_i denotes the category at hierarchy level l , ordered from c_1 very broad, to c_L a very fine category. For each path we construct the set of increasingly specific sub-paths: $\mathcal{P}(p_i) = \{(c_1), (c_1, c_2), \dots, (c_1, \dots, c_L)\}$. For two documents v_i and v_j each shared subpath contributes 0.75 to the weight of their hierarchical edge. Thus, the edge weight increases with the amount of their shared hierarchy: $w_e = 0.75 \cdot |\mathcal{P}(p_i) \cap \mathcal{P}(p_j)|$.
- $l_e = \text{shared categorical fields}$: If two documents share the same categorical field entry, such as author or affiliation, the edge receives a weight of $w_e = 0.5$.

Importantly if one node shares the same attribute, such as hierarchical path or categorical field with more than 50 other nodes, this relation does not create any edge, since sharing this relation is assumed to then be too generic or not important enough to give enough relevance signal at the retrieval stage. Let f be the expansion factor and k the number of top nodes we retain as candidates. Then the initial retrieval of our SAGE-inspired method selects the top $f \cdot k$ nodes as seed nodes. In the current settings $f = 2$. This helps in finding initially unrelated nodes which would otherwise not appear in the top-k, but could be related to a very relevant node. Retrieval is standard dense retrieval with the embedding model all-MiniLM-L6-v2.

In the second stage of retrieval each seed node is expanded in its own domain graph $G^{(d)}$ to their one hop neighbors which altogether form a set of nodes $\tilde{V} = S_q \cup N_q$ across graphs. S_q is the set of seed nodes, whereas N_q is the set of additional nodes retrieved which were not originally seed nodes corresponding to the query q . All nodes $\tilde{v}_j \in \tilde{V}$ then receive a new score F_j based on the first dense score and reweighting of the edge weights:

$$F_j = \mathbb{1}_{\{j \in S_q\}} \cdot s_j + \sum_{i \in S_q} \sum_{e \in E(i, j)} \lambda \cdot s_i \cdot \bar{w}_e \cdot \gamma_j$$

with

$$\gamma_j = \begin{cases} \max(0, s_j), & \tilde{v}_j \in S_q \\ 1, & \tilde{v}_j \in N_q \end{cases}$$

and

$$\bar{w}_e = \frac{w_e}{\sum_{e' \in E(i)} w_{e'} + \epsilon}$$

s_j represents the score of node $\tilde{v}_j$ and λ a factor on how important graph relations are to be considered, in the current setting $\lambda = 0.15$. Normalization of edge weights is important so nodes with high degree do not dominate the score. $\gamma_j = 1$ is set for nodes which were not in the initial ranking to allow structurally related documents not selected by the initial retriever to enter the candidate set based on graph relevance signal. After the calculation of the final scores the nodes are reranked based on the new scores and the final top-k nodes returned.

4.3 GNNRet-inspired method

This method is based on graph neural refinement and inspired by GNNRet introduced by Li et al. (2024). It starts by building an offline graph based on shared entities and labels over all dataset entries: two entries, represented by nodes, are connected if they share at least one entity (person, organization, project, or topic) or label. Document and query representations are frozen Sentence Transformer embeddings (all-MiniLM-L6-v2).

At inference time, GNNRet initializes a distance score for every document as $h_i^{(0)} = 1 - \cos(\text{doc}_i, \text{query})$ and then propagates these scores through the graph for L rounds (default $L = 5$). In each round, the K documents with the currently lowest distance (i.e. the most query-relevant so far) are selected as active seed nodes, which then send their scores to their neighboring nodes. Each destination node that receives at least one message updates its score by blending its own current distance with the minimum incoming message, controlled by a learned, per-round scalar mixing weight $\alpha^{(l)} \in [0, 1]$: $h_i^{(l)} = \alpha^{(l)} h_i^{(l-1)} + (1 - \alpha^{(l)}) \min_{j \rightarrow i} h_j^{(l-1)}$. Nodes that receive no message in a round keep their previous score unchanged. After L rounds, the final scores $h_i^{(L)}$ are inverted back into similarities $(1 - h_i^{(L)})$ and used to rank documents.

The α vector (one scalar per round, five parameters total) is the only learned component of the method, trained with a hinge loss on a small split of the available queries (10%, by default). For each training query, the loss compares the mean

propagated distance of the gold (relevant) documents, $d_y^{(L)}$, against the mean distance of the top- O non-gold documents after propagation, $d_o^{(L)}$: $\mathcal{L} = \max(0, \text{margin} + d_y^{(L)} - d_o^{(L)})$, optimized with SGD (default lr = 0.01, margin = 0.1, 10 epochs) and α clamped to $[0, 1]$ after every step.

4.4 GAT-inspired method

This method shares GNNRet’s core idea: propagate relevance scores over a metadata-derived document graph rather than ranking by embedding similarity alone. It replaces the scalar mixing weight $\alpha^{(l)}$ with a learned attention mechanism that decides, per edge, how much weight a neighbor’s score should receive. The graph is built the same way as GNNRet’s (pairs of directed edges representing shared entities and labels), but is augmented with explicit citation edges for papers: if a paper cites or is cited by another corpus document, that relationship is added as a directed edge on top of the entity/label graph. No distinction is made between edge types. Unlike GNNRet’s multi-round propagation, GAT performs a single attention-weighted aggregation step. This is deliberate: GNNRet’s per-round weight $\alpha^{(l)}$ is a single bounded scalar, so stacking several barely raises training risk, whereas GAT’s attention score already comes from a small MLP with four learned parameters per step. Chaining multiple such rounds would compound that added capacity and make training far more sensitive to initialization and learning rate.

For every edge $(i \rightarrow j)$ in the graph (including a self-loop for every node) an attention score is computed from the current distance values of the source and destination nodes, (h_i, h_j) , by a small two-layer MLP with a LeakyReLU nonlinearity: $e_{ij} = W_2 \sigma(W_1[h_j, h_i] + b_1) + b_2$, with a hidden dimension of 8 (default). These raw scores are normalized with a softmax over all edges arriving at the same destination node and the destination’s new score is the attention-weighted sum of its neighbors’ (and its own) current scores: $h_i^{\text{new}} = \sum_{j \in \mathcal{N}(i) \cup \{i\}} a_{ij} h_j$, $a_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp(e_{ik})}$. The resulting scores are inverted into similarities and used to rank documents. Training follows the same hinge-loss objective as GNNRet, comparing the mean propagated distance of gold documents against the top- O non-gold candidates, but here the learned parameters are the four attention weight matrices/biases rather than a handful of scalars.

5 Experimental Setup

The evaluation is performed on the aligned and enriched retrieval corpus, where documents and queries are represented in a common multi-field schema. The benchmark compares multiple retrieval methods across the same set of query–candidate pairs and uses a consistent ranking and aggregation procedure to ensure fair comparison.

5.1 Evaluation Metrics

The benchmark uses common retrieval measures that capture ranking quality, relevance ordering and retrieval completeness:

- **MRR** (Mean Reciprocal Rank): measures how early the first relevant document appears in the ranking. Higher values mean relevant results are found more quickly (Voorhees and Tice, 2000).
- **NDCG@k** (Normalized Discounted Cumulative Gain at cutoff k): evaluates ranking quality by rewarding relevant documents appearing higher in the top-k list, with diminishing weight for lower positions (Järvelin and Kekäläinen, 2002).
- **Precision@k**: measures the fraction of retrieved documents in the top-k that are relevant, reflecting retrieval accuracy within the selected cutoff (Blair, 1979).
- **MAP@k** (Mean Average Precision at cutoff k): computes, for each query, the mean of Precision@r at the ranks $r \leq k$ where relevant items appear and then averages this value over all queries. It emphasizes the accuracy of ranking relevant documents early (Voorhees and Tice, 2000).
- **Recall@k**: measures the fraction of all relevant candidates that appear in the top-k ranking, reflecting retrieval completeness (Blair, 1979).

These metrics are computed for each query and then averaged across the evaluation set to compare methods robustly.

5.2 Benchmark Workflow

The benchmark follows a structured evaluation pipeline. First, the pipeline loads the unified document corpus and relevance judgments from the

preprocessed dataset and exclude documents without structured metadata to ensure all methods can leverage the shared schema. Next, we construct method-specific indexes or representations (e.g., sparse indexes, dense embeddings, or multi-field vectors) over the candidate documents. For each query, we retrieve and rank candidates by computing similarity scores between the query representation and the document representations, producing a top- k list. We then evaluate the ranked results against the ground-truth set using the selected metrics.

6 Results & Discussions

The following section presents the results of our project, along with a discussion of the findings and their implications.

6.1 Quantitative Comparison

Figure 1 compares all four experimental methods against the baselines (BM25 and dense embedding). All six methods were run head-to-head on the same enriched dataset (78,002 documents, 1,000 queries) and evaluated at $k = 10$.

The four methods form a clear, ordered progres-

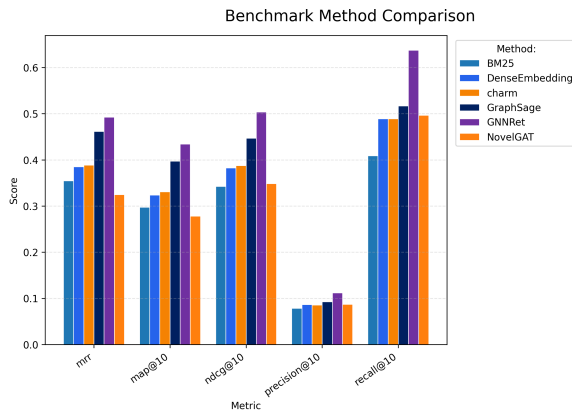


Figure 1: MRR, MAP@10, NDCG@10, Precision@10 and Recall@10 for the baselines and all four experimental methods on the enriched dataset.

sion that tracks how much of the document’s structure each one actually uses:

- **CHARM** (metadata-aware, no graph) is within $\pm 2\%$ from the better performing baseline on every metric
- **GraphSAGE** (offline graph, online neighbor expansion) is the first method to show a clear, consistent lift

- **GNNRet** (learned multi-round propagation over the same graph) is the strongest method across the board
- **GAT** (attention-weighted propagation) is the outlier: with this configuration it underperformed the baseline on some of the metrics. Unlike the other three methods, its performance varied noticeably with training configuration.

One value in Figure 1 stands out regardless of method: Precision@10 is low in absolute terms for every method, ranging from about 0.08 to 0.11. This is not a sign that the retrievers are performing poorly, it is a ceiling effect of the ground truth. Most queries in our dataset have only one or two annotated relevant documents (66% have exactly one, with an average of 2.06 per query) and since Precision@10 divides the number of relevant documents retrieved by $k = 10$, a query with a single relevant document caps out at $1/10 = 0.1$ even for a perfect retriever that ranks it first.

6.2 Key Findings

Metadata and hierarchy substantially improve retrieval on semi-structured data, but only when used relationally: GraphSAGE and GNNRet, which convert categorical, hierarchical and entity metadata into graph edges, improve every evaluated metric by 15-31% over the dense baseline. Flattening that same metadata into text is not an effective substitute for this structure: CHARM, which encodes it as additional weighted text fields, is statistically indistinguishable from the metadata-blind baseline, confirming that the value of hierarchy lies in the relations it defines rather than in the information it contains. Among the graph-based methods, learned propagation improves on static expansion: GNNRet adds a further 6-10% over GraphSAGE on the same underlying graph by learning how strongly to trust each propagation round, showing that how the graph is used matters as much as whether it is used. The attention-based variant, GAT, illustrates the limits of that flexibility: its performance appears to depend heavily on configuration, indicating that more expressive propagation rules require more careful tuning to reliably realize the graph’s benefit.

6.3 Strengths and Limitations

A key strength of this evaluation is that all methods were compared under a single, consistent proto-

col: the same corpus, query set and metrics, making the observed differences attributable to how each method uses structure rather than to implementation details. Because the two datasets we chose differ in subject matter, metadata schema and the nature of their ground truth relations, an improvement that holds across both is more likely to transfer to other semi-structured domains, such as ticketing systems, product catalogs, legal case repositories, or patent databases.

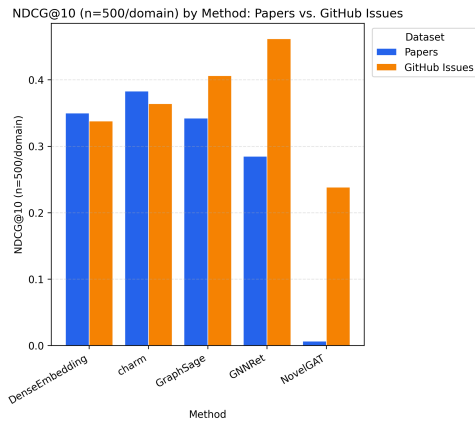


Figure 2: NDCG@10 by method separated by data source

However, because explicit issue links dominate our dataset numerically, the aggregate scores in Figure 1 skew more heavily toward GitHub-style queries than toward paper-style queries, and a benchmark run with a different domain mix could shift the reported margins (as shown in Figure 2). Similarly, the ground-truth sparsity discussed above means that a small number of queries linked to many documents can noticeably move the aggregated metrics (Precision@10 in particular is sensitive to this distribution rather than to retrieval quality alone). Finally, the quality of the metadata is not uniform across documents: GitHub issue labels are provided directly, whereas the hierarchical fields for papers were derived via LLM-based enrichment and Semantic Scholar metadata that is not available for every corpus entry, so part of the observed variation is inherited from differences in metadata completeness and quality between the two domains rather than from the retrieval methods themselves.

6.4 Comparison with the Literature

Relating each method back to its original formulation helps explain where our results diverge.

Our CHARM result differs most from its source. Freymuth et al. (2025) report gains from learned hierarchical attention and re-ranking, while our simpler version uses only separate field embeddings with static weights. Its near-zero effect suggests that CHARM’s gains come mainly from the components we omitted.

The graph-based methods align well with prior work. Both Titiya et al. (2026) and Li et al. (2024) show that graph structure improves retrieval and that learned propagation outperforms static expansion. Our results reproduce this ordering (GraphSAGE over the baseline and GNNRet over GraphSAGE) across two different domains, supporting the generality of the approach.

GAT is the main exception. While Veličković et al. (2018) show that attention can improve fixed aggregation, our GAT underperforms GraphSAGE. This likely reflects our limited training data, fixed hyperparameters, and single propagation step rather than a fundamental limitation of attention. More expressive propagation methods may simply require more careful tuning to perform well.

7 Conclusion

This project set out to determine how the categorical, hierarchical and relational information already present in semi-structured documents can improve retrieval. The answer, evaluated jointly on a combined corpus of GitHub issues and scientific papers, is a clear yes, with one important caveat: how the metadata is used matters. Simply incorporating metadata into the text representation (CHARM-inspired) has little to no effect on retrieval performance. In contrast, using the same metadata to build a document graph (GraphSAGE, GNNRet) improves all ranking metrics by 15-31% compared with a strong dense-retrieval baseline. Learning how to propagate relevance across that graph, rather than relying on static neighborhood expansion, provides a further, consistent improvement in GNNRet. Meanwhile the more expressive attention-based alternative, GAT, shows that increasing model flexibility introduces additional sensitivity to training configuration that must be managed carefully to be worthwhile.

Declaration of AI/LLM Usage

During this project we made use of AI tools and large language models. The following outlines which tools were used, for what purpose and to

what extent.

- **Tool(s) used:** ChatGPT, Claude Chat, GitHub Copilot
- **Purpose of use:** obtaining an initial overview of the project, idea generation, support for project planning and tracking, code scaffolding, help with bug fixing and refactoring, deepening our understanding of newly learned concepts, paraphrasing, literature discovery and documentation generation.
- **Extent of use:** Chatbots such as ChatGPT and Claude Chat served as general-purpose assistants throughout the project. They were mainly used to speed up the initial exploration of unfamiliar topics and to deepen our understanding of newly acquired knowledge. GitHub Copilot was primarily used for code scaffolding and for drafting smaller code fragments during implementation. In addition, AI tools were used to propose timelines and concrete next steps and to support status reviews during project execution.

References

- Anirudh Ajith, Mengzhou Xia, Alexis Chevalier, Tanya Goyal, Danqi Chen, and Tianyu Gao. 2024. [LitSearch: A Retrieval Benchmark for Scientific Literature Search](#). *arXiv preprint*. ArXiv:2407.18940 [cs].
- Zakarea Alshara. 2022. [Pi-link: A ground-truth dataset](#).
- Zakarea Alshara, Anas Shatnawi, Hamzeh Eyal-Salman, Abdelhak-Djamel Seriai, and Maad Shatnawi. 2023. [PI-Link: A Ground-Truth Dataset of Links Between Pull-Requests and Issues in GitHub](#). *IEEE Access*, 11:697–710.
- James Bergstra and Yoshua Bengio. 2012. [Random search for hyper-parameter optimization](#). *The Journal of Machine Learning Research*, 13(null):281–305.
- David C. Blair. 1979. [Information retrieval](#), 2nd ed. c.j. van rijnsbergen. london: Butterworths; 1979: 208 pp. price: \$32.50. *Journal of the American Society for Information Science*, 30(6):374–375.
- Niklas Freymuth, Dong Liu, Thomas Ricatte, and Saab Mansour. 2025. [Hierarchical Multi-field Representations for Two-Stage E-commerce Retrieval](#). *arXiv preprint*. ArXiv:2501.18707 [cs.IR].
- Princeton NLP Group. 2026. [princeton-nlp/LitSearch · Datasets at Hugging Face](#).
- Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. [Inductive Representation Learning on Large Graphs](#). In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Kalervo Järvelin and Jaana Kekäläinen. 2002. [Cumulated gain-based evaluation of ir techniques](#). *ACM Trans. Inf. Syst.*, 20(4):422–446.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. [Dense Passage Retrieval for Open-Domain Question Answering](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.
- Rodney Kinney, Chloe Anastasiades, Russell Authur, Iz Beltagy, Jonathan Bragg, Alexandra Buraczynski, Isabel Cachola, Stefan Candra, Yoganand Chandrasekhar, Arman Cohan, Miles Crawford, Doug Downey, Jason Dunkelberger, Oren Etzioni, Rob Evans, Sergey Feldman, Joseph Gorney, David Graham, Fangzhou Hu, and 29 others. 2025. [The semantic scholar open data platform](#). *Preprint*, arXiv:2301.10140.
- Thomas N. Kipf and Max Welling. 2017. [Semi-Supervised Classification with Graph Convolutional Networks](#). *arXiv preprint*. ArXiv:1609.02907 [cs.LG].
- Zijian Li, Qingyan Guo, Jiawei Shao, Lei Song, Jiang Bian, Jun Zhang, and Rui Wang. 2024. [Graph Neural Network Enhanced Retrieval for Question Answering of LLMs](#). *arXiv preprint*. ArXiv:2406.06572 [cs.CL].
- Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.
- Nils Reimers and Iryna Gurevych. 2019. [Sentence-bert: Sentence embeddings using siamese bert-networks](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.
- Stephen Robertson and Hugo Zaragoza. 2009a. The probabilistic relevance framework: Bm25 and beyond. *Foundations and trends® in information retrieval*, 4(1-2):1–174.
- Stephen Robertson and Hugo Zaragoza. 2009b. [The probabilistic relevance framework: Bm25 and beyond](#). *Found. Trends Inf. Retr.*, 3(4):333–389.
- Hinrich Schütze, Christopher D Manning, and Prabhakar Raghavan. 2008. *Introduction to information retrieval*, volume 39. Cambridge University Press Cambridge.
- Prasham Titiya, Rohit Khoja, Tomer Wolfson, Vivek Gupta, and Dan Roth. 2026. [SAGE: Structure Aware Graph Expansion for Retrieval of Heterogeneous Data](#). *arXiv preprint*. ArXiv:2602.16964 [cs.IR].

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. [Graph attention networks](#). *Preprint*, arXiv:1710.10903.

Ellen M. Voorhees and Dawn M. Tice. 2000. [The TREC-8 question answering track](#). In *Proceedings of the Second International Conference on Language Resources and Evaluation (LREC'00)*, Athens, Greece. European Language Resources Association (ELRA).

Shirley Wu, Shiyu Zhao, Michihiro Yasunaga, Kexin Huang, Kaidi Cao, Qian Huang, Vassilis N. Ioannidis, Karthik Subbian, James Zou, and Jure Leskovec. 2024. [Stark: Benchmarking llm retrieval on textual and relational knowledge bases](#). *Preprint*, arXiv:2404.13207.

Zhichao Xu, Fengran Mo, Zhiqi Huang, Crystina Zhang, Puxuan Yu, Bei Wang, Jimmy Lin, and Vivek Sriku-mar. 2026. [A survey of model architectures in information retrieval](#). *Preprint*, arXiv:2502.14822.

Raquib Bin Yousuf, Shengzhe Xu, Mandar Sharma, Andrew Neeser, Chris Latimer, and Naren Ramakrishnan. 2026. Utilizing metadata for better retrieval-augmented generation. In *European Conference on Information Retrieval*, pages 305–319. Springer.

A Contribution and Work Split

Table 1 shows the main person responsible for each part, but everyone contributed at least some share to each package. Names are sorted alphabetically and the ordering does not indicate the amount of work.

Work Item	Responsibility
GitHub Issues Dataset	Olga
Scientific Papers Dataset	Michaela
Common Schema Design	Dennis
Dataset Alignment	Dennis
Metadata Enrichment	Michaela, Olga
Hierarchy Enrichment	Michaela, Olga
Baseline	Dennis
CHARM	Dennis
SAGE	Michaela
GNNRet	Olga
GAT	Everyone
Literature Research	Everyone
Benchmark Pipeline	Everyone
Documentation	Everyone
Poster	Everyone
Report	Everyone

Table 1: Detailed work split across project components.